THE OPEN SOURCE AUDIT

A Complete Academic Report on the Linux Kernel

VITyarthi | Open Source Software

TANISHKA SHARMA- 24BCE11422

Software	Linux Kernel
Licence	GNU General Public License v2 (GPLv2)
Sections	A: Origin & Philosophy | B: Linux Footprint | C: Ecosystem | D: Comparison | Scripts 1–5
Submitted	31 March 2026

Introduction

This report is a complete Open Source Audit of the Linux kernel — the software that runs 96% of the world's web servers, every computer on the Top 500 supercomputer list, and every Android phone ever shipped. Produced as part of the VITyarthi Open Source Software course, it follows the ten-day timeline in Section 2. It covers the kernel's origin and philosophy (Part A), its footprint on a live Ubuntu 24.04.4 system (Part B), the surrounding FOSS ecosystem (Part C), a critical comparison against Microsoft Windows Server (Part D), and five original shell scripts (Section 4).

All commands and output in Parts B and C were run live on Ubuntu 24.04.4 LTS (kernel 4.4.0, GCC 13.3.0) and are reported verbatim. The five scripts were written, tested, and confirmed working on the same system.

Section 2 — Project Timeline

The following ten-day schedule guided the development of this report. Each day has a single focus, a clear set of tasks, and a concrete deliverable. The timeline is a guide, not a rigid rule — what it

enforces is that scripts are not left to the final stretch, and each section is drafted before the next begins.

Day	Focus	Tasks	Deliverable
1	**Setup**	Choose software. Research origin story. Read the licence. Set up Linux VM or lab system. Install chosen software.	Software choice confirmed
2	**Philosophy**	Write Part A1 — origin story. Notes on what problem it solved and for whom.	Draft A1
3	**Licence & Ethics**	Write Part A2 — licence analysis. Write Part A3 — ethical reflection. Own thinking, not summaries.	Draft A2 + A3
4	**Linux Footprint**	Install software on Linux. Document directories, user, permissions. Write Part B with screenshots.	Draft Part B + screenshots
5	**Scripts 1 & 2**	Write and test Script 1 (System Identity) and Script 2 (Package Inspector). Add to report with explanations.	Scripts 1–2 working
6	**Scripts 3 & 4**	Write and test Script 3 (Disk Auditor) and Script 4 (Log Analyser). Test with a real log file.	Scripts 3–4 working
7	**Ecosystem**	Write Part C — FOSS ecosystem map. Research dependencies and community.	Draft Part C
8	**Comparison + S5**	Write Part D — comparison table and verdict. Write and test Script 5 (Manifesto Generator).	Draft Part D + Script 5
9	**Report polish**	Compile full report. Write introduction and conclusion. Add screenshots. Check page count (12–16 pages).	Complete draft
10	**Submission**	Final review. Submit report + 5 .sh files.	Submitted ✓

Part A — Origin and Philosophy

A1. The Problem Linux Was Created to Solve

To understand why Linux exists, you have to go back to 1991 and into the situation of a 21-year-old Finnish computer science student named Linus Torvalds. He was studying at the University of Helsinki, and he had a problem that was both financial and deeply technical.

At the time, if you wanted a serious Unix-like operating system — the kind used by universities, engineers, and professionals — your options were expensive. AT&T's Unix cost thousands of dollars in licensing fees. Even MINIX, a teaching-oriented Unix clone built by Professor Andrew Tanenbaum, was restrictive: you could read its source code, but you could not modify and redistribute it freely. MINIX was deliberately kept small because Tanenbaum wanted it to illustrate textbook concepts, not become a production system.

Torvalds wanted something more. He had just bought an Intel 386-based PC and wanted to use it the way professionals used workstations: multitasking, a real shell, serious software. He could not afford a proper Unix licence, and MINIX would not let him go far enough.

So in April 1991 he began writing his own kernel — the core of an operating system that manages hardware, memory, and processes. On 25 August 1991 he posted a now-famous message to the comp.os.minix newsgroup, describing it as ‘just a hobby, won’t be big and professional like gnu.’ He was wrong about that.

> **The famous post (25 Aug 1991):** "I’m doing a (free) operating system (just a hobby, won’t be big and professional like gnu) for 386(486) AT clones. This has been brewing since april, and is starting to get ready." — Linus Torvalds

What made Torvalds’s decision historic was not just that he built something — it was that he shared it. He posted the source code publicly and invited others to contribute. Within months, programmers around the world were sending patches, drivers, and fixes. What began as a student’s weekend project became one of the largest collaborative engineering efforts in history.

Linux did not arise from a corporation identifying a market gap. It arose from frustration — from wanting something that should exist and finding that proprietary barriers stood in the way. Keeping it proprietary would have defeated the entire purpose. Torvalds also had a practical reason for sharing: by opening it up he got something no solo programmer can achieve — thousands of contributors improving it simultaneously.

Richard Stallman’s GNU project, started in 1983, had already produced essential free software tools — compilers, editors, libraries — but lacked a kernel. Linux provided exactly that missing piece. The GNU tools and the Linux kernel together form GNU/Linux: a complete, free, Unix-like operating system. Torvalds did not build Linux in a vacuum; he completed a decade of prior community effort.

A2. The Licence — What It Actually Says

Linux is distributed under the GNU General Public Licence version 2 (GPLv2) — one of the most significant licences ever written, revealing a philosophy as much as a legal framework.

The four freedoms of free software, as defined by the Free Software Foundation:

- Freedom 0 — The freedom to run the program for any purpose.
- Freedom 1 — The freedom to study how the program works and modify it.
- Freedom 2 — The freedom to redistribute copies so you can help others.
- Freedom 3 — The freedom to distribute copies of your modified versions.

Linux’s GPLv2 grants all four freedoms. Anyone can download the kernel source, run it on any machine, modify it, and redistribute both original and modified versions.

Can a company take Linux, modify it, and sell it without sharing changes? No. The GPLv2 contains a ‘copyleft’ clause: if you distribute a modified version, you must make the source code of your modifications available under the same licence. A company can sell a product built on Linux — Red Hat does exactly this — but it cannot keep its modifications secret.

GPL vs MIT: The MIT licence is permissive — it allows taking the code, modifying it, incorporating it into a proprietary product, and redistributing without sharing changes. The GPL demands the same freedoms be preserved downstream. For something intended to remain free forever, GPL is the right choice. For maximum adoption where corporate use in closed products is acceptable, MIT offers simpler terms.

'Free as in free beer' vs 'free as in freedom': Free beer means zero cost. Free as in freedom means you have rights over the software — to read it, modify it, share it. Linux is both. Red Hat charges for enterprise support, not for the software itself — entirely consistent with the GPLv2.

A3. The Ethics of Open Source

Should all software be open source?

The case for it is compelling: when a hospital runs on closed-source software and a critical bug appears, neither staff nor independent researchers can fix it without the vendor's cooperation. When an election system is proprietary, voters cannot verify its integrity. Transparency makes software more trustworthy, more secure, and more durable.

The case against is also worth taking seriously. Writing software well requires sustained human effort that has to be funded. Many useful applications — specialist medical tools, complex financial systems — are built by companies that need revenue to survive. A reasonable middle position: infrastructure software should be open source because the entire ecosystem depends on it being trustworthy. Application software might remain proprietary where development cost is high and the user base is small.

Is it ethical for companies to profit from free community labour?

Google's Android, Amazon's AWS, and Meta's servers all run on the Linux kernel. These are trillion-dollar enterprises whose foundations depend on community work. Red Hat contributes significant engineering back to the kernel and makes money from support contracts — a fair exchange. But a company extracting enormous value while contributing little back feels like a betrayal of spirit, even if it is within the letter of the law.

Standing on the shoulders of giants: In software this is not a metaphor — it is literal. Every Linux program runs on system calls defined by Unix conventions from the 1970s. Open source makes this debt visible and repayable. Linux spawned Android. Apache powered the early commercial internet. Python became the dominant language of data science. Open foundations enable innovation at a scale no proprietary platform could match.

Part B — Linux Footprint on a Linux System

> **Lab environment:** Ubuntu 24.04.4 LTS (Noble Numbat), Kernel 4.4.0. All commands below were run and verified live on this system.

B1. Installation Method

The Linux kernel is delivered as .deb packages via apt on Ubuntu/Debian systems. Confirmed available versions on this machine:

```
$ apt-cache policy linux-image-generic
linux-image-generic:
  Candidate: 6.8.0-100.100
  Available: noble-updates/main, noble-security/main
```

Installation via apt:

```
sudo apt update
sudo apt install linux-image-generic
# Pulls in: linux-image-6.8.0-100-generic, linux-modules,
#           linux-firmware, initramfs-tools, grub-pc, udev
sudo reboot   # new kernel becomes active after reboot
```

Compiling from source:

```
git clone https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git
cd linux && git checkout v6.8
make menuconfig           # interactive feature configuration
make -j$(nproc)           # build on all CPU cores (~30-90 min)
sudo make modules_install # install .ko files to /lib/modules/
sudo make install         # install kernel image to /boot/
sudo update-grub          # update bootloader
```

Compilers confirmed present: GCC 13.3.0, GNU Binutils 2.42, GNU Make 4.3, Perl 5.38, Python 3.12.3.

B2. Key Directories

The Linux kernel occupies several distinct directories across the filesystem, each serving a defined purpose under the Filesystem Hierarchy Standard (FHS). All entries below were verified live.

Directory	Purpose and live evidence
/boot	Kernel image (vmlinuz), initrd, and GRUB config. Populated by apt or make install during kernel installation.
/proc	Virtual filesystem in memory. /proc/version confirmed: 'Linux version 4.4.0'. /proc/cpuinfo, /proc/meminfo, /proc/loadavg all live and readable.
/proc/sys	Kernel tunables. Confirmed live: hostname=runsc, pid_max=65536, ostype=Linux. Readable by all; writable only by root.
/sys	Hardware and driver info (sysfs). Confirmed: /sys/block, /sys/bus, /sys/class, /sys/devices, /sys/kernel, /sys/module, /sys/power.
/lib/modules/	Loadable kernel modules (.ko files) by kernel version. Contains thousands of files on a full install.

Directory	Purpose and live evidence
/etc	Kernel-adjacent config: /etc/fstab, /etc/sysctl.conf, /etc/default/grub. 137 items confirmed on this machine.
/var/log	Runtime logs: /var/log/kern.log, /var/log/syslog. Confirmed: /var/log/dpkg.log has 8,185 lines of real history.

B3. User and Group — Who Runs the Kernel?

The Linux kernel runs in CPU Ring 0 — kernel mode — a hardware-enforced privilege level entirely separate from the Linux user/group permission system. Confirmed on this machine: uid=0(root) gid=0(root). The 'nobody' overflow UID ceiling is confirmed at 65534.

This matters for security because Linux uses the user/group system to isolate processes. A web server running as root that gets compromised gives an attacker full control. The same server running as www-data limits the blast radius enormously. The kernel enforces this through capability bits such as CAP_NET_ADMIN and CAP_SYS_MODULE.

Security feature confirmed active: Yama LSM (ptrace_scope=1) — restricts process tracing to prevent a compromised process reading secrets from sibling processes' memory.

B4. Service Management

The kernel cannot be started or stopped as a service — it is the running system. Once running, it hands control to systemd (PID 1). Confirmed: /sbin/init → ../lib/systemd/systemd, systemd version 255.

```
journalctl -k                        # kernel messages only
dmesg | tail -30                     # raw kernel ring buffer
systemctl status systemd-udevd        # hardware device manager
systemctl restart systemd-udevd
lsmod                                # list loaded kernel modules
sudo modprobe bluetooth              # load a module
sudo rmmod bluetooth                 # unload a module
sysctl kernel.pid_max                # read tunable: 65536
sudo sysctl -w net.ipv4.ip_forward=1 # write tunable
sudo sysctl -p /etc/sysctl.conf      # reload from config file
```

B5. Update Model — How Security Patches Reach Your Machine

When a vulnerability is found, the fix is published to LKML and committed to git.kernel.org — public from the moment it lands. Distribution teams backport it, test it, and push it to their package repositories. Confirmed sources on this machine: ubuntu.sources covers noble-updates and noble-security.

```
sudo apt update    # fetch updated package lists
sudo apt upgrade   # download and install all updates including kernel patches
```

```
sudo reboot        # new kernel becomes active
# Canonical Livepatch patches the running kernel in memory without rebooting
```

The key advantage over proprietary systems: when Linux is patched, the diff is public, peer-reviewed, and independently auditable. Vulnerabilities receive CVE numbers; Ubuntu Security Notices cross-reference each kernel update to the CVEs it addresses.

Part C — The FOSS Ecosystem

C1. What Linux Depends On

Linux depends on a precise set of tools to be built, and an entire GNU ecosystem to be useful once running. All versions below were confirmed live on Ubuntu 24.04.

Dependency	Version	Role
GCC	13.3.0	Compiles the kernel's C source into machine code. Primary build tool since 1991.
GNU Binutils	2.42	Assembler (as) and linker (ld) that combine compiled objects into the single vmlinuz binary.
GNU Make	4.3	Drives the entire build process via the kernel's Makefile system.
Perl	5.38	Used in scripts/ directory. The Kconfig build configuration system relies on Perl.
Python	3.12.3	Kernel testing (kunit), documentation generation, and build tooling.
glibc	2.39	C standard library — bridges all userspace programs to the kernel via 373 system calls.
GNU Coreutils	9.4	Provides ls, cat, cp, chmod and 100+ commands. Without these, a running kernel has no tools.
systemd	255	Init system (PID 1). First userspace process; manages all services and mounts.
git	2.43	Written by Torvalds in 2005 to manage Linux development. Now the global standard for VCS.
bash	5.2.21	GNU shell — the first interactive program most users run on a Linux system.

C2. What Linux Has Enabled

Android: Google forked the Linux kernel in 2007 and adapted it for mobile hardware. Android runs on approximately 72% of all mobile devices worldwide. Every Android phone is running a Linux kernel.

Containers (Docker, Kubernetes): An application of two kernel features. Namespaces (confirmed: ipc, mnt, net, pid, user, uts) give a process an isolated view of the system. Cgroups (confirmed: cpu, memory, pids) limit resource consumption. Docker is a management tool wrapping these two kernel primitives.

Supercomputing: As of November 2024, every one of the world's 500 most powerful computers runs Linux — including Frontier (the first exascale computer), Aurora, and LUMI.

The web: The LAMP stack (Linux, Apache, MySQL, PHP) powered the commercial internet from the mid-1990s. AWS, Google Cloud, and Azure all use Linux as their primary host OS. Microsoft reports over 50% of Azure VMs run Linux.

eBPF: Introduced in kernel 3.18 (2014), eBPF allows programs to be loaded into the running kernel and executed in response to system events. Cloudflare uses it to filter billions of malicious packets per second.

C3. Linux and the LAMP Stack

LAMP stands for Linux, Apache, MySQL, and PHP. Linux provides the TCP/IP networking stack that makes HTTP connections possible, the virtual filesystem that Apache uses to read files, the process scheduler that gives each request a CPU time slice, and the memory management that allows MySQL to allocate memory safely. When a browser makes a request, it travels through the Linux kernel's networking stack before Apache ever sees it.

C4. Community, Governance, and Who Is in Charge

The Linux Kernel Mailing List (LKML — linux-kernel@vger.kernel.org) is the primary communication channel, active since 1991, receiving approximately 10,000 messages per month at peak. All patches, disputes, and design decisions happen publicly here.

The governance structure is hierarchical but not corporate. At the top is Linus Torvalds. Below him are approximately 2,500 named subsystem maintainers. The release cycle runs 9–10 weeks: a 2-week merge window for new features, then release candidates rc1–rc8 for bug fixes, then a final tagged release.

The Linux Foundation (formed 2007) provides legal and financial infrastructure. Its 950+ member companies include IBM, Intel, Google, Samsung, Oracle, Qualcomm, and Microsoft. It employs Torvalds directly. Greg Kroah-Hartman maintains the LTS trees (6.6, 6.1, 5.15, 5.10, 5.4) with support windows to 2026.

Conferences: Linux Plumbers Conference, Open Source Summit, Kernel Recipes, and FOSDEM in Brussels (~8,000 attendees, free entry). An important characteristic: there is no roadmap. Each release is whatever was ready, reviewed, and merged in that cycle.

Part D — Open Source vs Proprietary: Critical Analysis

Linux's closest proprietary alternative is Microsoft Windows Server — the only other OS with comparable adoption across enterprise, cloud, and server deployments. The comparison below is honest and does not assume open source is automatically superior in every dimension.

Dimension	Linux (GPLv2)	Windows Server (Proprietary)
Cost	Free to download and run. Enterprise support via Red Hat (~$1,200–$7,000/yr) is optional, not mandatory.	~$1,069 (Standard) to ~$6,155 (Datacenter) per 16-core licence, plus Client Access Licences per connecting user.
Security — code audit	Full source is public. Any researcher can audit it. CVEs are disclosed and patched publicly. ~250 kernel CVEs in 2023.	Source is closed. Only Microsoft and vetted partners can audit the code. ~900+ CVEs in 2023. Patch Tuesday means weeks between disclosure and fix.
Support & reliability	Community support free but unguaranteed. Paid RHEL/Canonical support provides SLAs. Linux systems routinely run 500–1,000+ days without reboot; Livepatch avoids even that.	Microsoft Premier Support is expensive but accountable. Monthly Patch Tuesday reboots required — a real operational cost in 24/7 environments.
Freedom to modify	Complete. GPLv2 guarantees the right to read, modify, and redistribute. Google rewrote the scheduler for Android. Modifications must be shared if distributed.	None. The EULA prohibits reverse engineering, modification, or redistribution. You configure Windows within Microsoft's limits only.
Community control	Governed by Linus Torvalds and ~2,500 named maintainers. Linux Foundation provides oversight. No single company controls the roadmap.	Controlled entirely by Microsoft. Roadmap, pricing, and features decided internally. Users have no vote.
Overall verdict	Best for servers, cloud, embedded, research, and any environment requiring customisation, auditability, or cost control. 96% of web servers and 100% of supercomputers run Linux.	Better where Windows-native apps (Active Directory, Exchange, specific IIS/ASP.NET stacks) are a hard requirement, or staff are exclusively Windows-trained.

Conclusion and Deployment Recommendation

For any server, cloud, or infrastructure workload where there is a choice, Linux is the correct decision. The cost case alone is compelling — a medium-sized organisation running 50 servers

saves tens of thousands of dollars annually in licence fees. But cost is the least interesting reason. The deeper case is control and trust. When a critical vulnerability is found in Linux, the fix is published publicly on LKML, reviewed by independent experts, and shipped through apt within days. When a comparable vulnerability is found in Windows, users wait for Patch Tuesday, receive a binary they cannot inspect, and must trust Microsoft fixed the right thing correctly. For systems handling sensitive data — hospitals, banks, government services — the ability to independently audit the code is not a luxury. It is a fundamental requirement.

On contribution: the answer is yes. The kernel's process is demanding — patches are reviewed rigorously — but it is entirely open. Start by reading Documentation/process/ in the kernel source tree, find a subsystem with beginner-friendly tags in the MAINTAINERS file, and fix a small bug. Many contributors begin with compiler warnings or documentation. The barrier is effort, not permission — which is exactly what makes it worth attempting.

Section 4 — Shell Script Tasks

Five shell scripts were written, tested, and confirmed working on Ubuntu 24.04.4 LTS. Each is submitted as a separate .sh file and documented below with the complete annotated code, live terminal output, and an explanation of the shell scripting concepts demonstrated.

Script 1 — System Identity Report

Usage: bash script1_system_identity.sh

Displays a formatted welcome screen showing the Linux distribution name, kernel version, currently logged-in user, home directory, system uptime, current date and time, and the open-source licence covering the OS. Designed to run at login or as the opening step of a system audit.

Shell concepts demonstrated:

- Variables — STUDENT_NAME, SOFTWARE_CHOICE, LICENSE hold fixed string values at the top of the script, making them easy to customise without modifying the rest of the code.
- Command substitution $() — KERNEL=$(uname -r), USER_NAME=$(whoami), UPTIME=$(uptime -p), and CURRENT_DATE=$(date ...) each run a command and store its output. The output changes every time the script runs.
- Pipe and cut — grep '^PRETTY_NAME' /etc/os-release | cut -d"" -f2 finds the distro name line and extracts the value between quote characters.
- Formatted echo — multi-line echo statements with consistent spacing produce a readable, bordered output panel.

Complete annotated script:

```
#!/bin/bash
# Script 1: System Identity Report
```

```
# Author: TANISHKA SHARMA | Course: Open Source Software
# Purpose: Display a formatted welcome screen with live system info

# --- Variables: customise these ---
STUDENT_NAME=Tanishka Sharma
SOFTWARE_CHOICE="Linux Kernel"
LICENSE="GNU General Public License version 2 (GPLv2)"

# --- Command substitution: capture live system data ---
KERNEL=$(uname -r)                              # kernel version
USER_NAME=$(whoami)                             # logged-in user
USER_HOME=$(eval echo ~$USER_NAME)              # home directory
UPTIME=$(uptime -p)                             # human-readable uptime
CURRENT_DATE=$(date '+%A, %d %B %Y %H:%M:%S')  # formatted date/time

# --- Read distro name from /etc/os-release ---
DISTRO=$(grep '^PRETTY_NAME' /etc/os-release | cut -d'"' -f2)

# --- Display formatted output using echo ---
echo "================================================================"
echo "      OPEN SOURCE AUDIT --- System Identity Report"
echo "================================================================"
echo "  Student      : $STUDENT_NAME"
echo "  Software     : $SOFTWARE_CHOICE"
echo "================================================================"
echo "  Distribution : $DISTRO"
echo "  Kernel       : $KERNEL"
echo "  User         : $USER_NAME"
echo "  Home Dir     : $USER_HOME"
echo "  Uptime       : $UPTIME"
echo "  Date/Time    : $CURRENT_DATE"
echo ""
echo "  License  : $LICENSE"
echo "  This OS is free software: study, modify, and redistribute"
echo "  it under the terms of the GPLv2."
echo "================================================================"
```

Live terminal output (Ubuntu 24.04.4 LTS, confirmed):

```
================================================================
      OPEN SOURCE AUDIT --- System Identity Report
================================================================
  Student      : Tanishka Sharma
  Software     : Linux Kernel
================================================================
  Distribution : Ubuntu 24.04.4 LTS
  Kernel       : 4.4.0
  User         : root
  Home Dir     : /root
  Uptime       : up 1 minute
  Date/Time    : Saturday, 28 March 2026 10:35:27

  License  : GNU General Public License version 2 (GPLv2)
  This OS is free software: study, modify, and redistribute
  it under the terms of the GPLv2.
================================================================
```

Script 2 — FOSS Package Inspector

Usage: bash script2_package_inspector.sh

Checks whether a named package is installed on the system using dpkg. If installed, displays the version, summary, and licence info. A case statement then matches the package name to a philosophy statement regardless of installation status. Demonstrated with 'bash' (installed, v5.2.21) and 'linux-base'.

Shell concepts demonstrated:

- if-then-else — tests whether dpkg -l returns a line beginning with 'ii' (installed). The else branch handles the not-installed case gracefully with an install hint.
- case statement — matches $PACKAGE against a list of patterns. Each) is a pattern; ;; terminates that branch; *) is the catch-all default. More readable than a long elif chain.
- Pipe with grep — dpkg -l "$PACKAGE" | grep -q '^ii' sends dpkg output into grep, which returns exit code 0 on match. The if statement acts on that exit code.
- dpkg-query — extracts specific package metadata fields such as ${Version} and ${Description}.

Complete annotated script:

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Tanishka Sharma Course: Open Source Software

PACKAGE="linux-base"   # change to your target package

echo "============================================"
echo " FOSS Package Inspector: $PACKAGE"
echo "--------------------------------------------"

# --- if-then-else: check installation status via dpkg ---
if dpkg -l "$PACKAGE" 2>/dev/null | grep -q '^ii'; then
    echo " STATUS  : INSTALLED"
    VERSION=$(dpkg-query -W -f='${Version}' "$PACKAGE" 2>/dev/null)
    DESCRIPTION=$(dpkg-query -W -f='${Description}' "$PACKAGE" | head -1)
    echo " Version : $VERSION"
    echo " Summary : $DESCRIPTION"
    echo " License : See /usr/share/doc/$PACKAGE/copyright"
else
    echo " STATUS  : NOT INSTALLED"
    echo " To install: sudo apt install $PACKAGE"
fi

echo "--------------------------------------------"
echo " Open Source Philosophy Note:"

# --- case statement: match package name to philosophy statement ---
case "$PACKAGE" in
    linux-base|linux-image-*)
        echo " Linux: freed computing from proprietary constraints."
        echo " Runs 96% of web servers and every supercomputer." ;;
    apache2|httpd)
        echo " Apache: the web server that built the open internet." ;;
    mysql-server|mysql)
        echo " MySQL: open source at the heart of millions of apps." ;;
    firefox)
        echo " Firefox: kept the web open when IE threatened it." ;;
    git)
        echo " Git: Torvalds built this to manage Linux itself." ;;
    python3|python)
        echo " Python: democratising programming for a generation." ;;
    *)   # default catch-all for any other package
        echo " '$PACKAGE': free to use, study, and share." ;;
esac
```

```
echo "==========================================="
```

Live terminal output (Ubuntu 24.04.4 LTS, confirmed):

```
===========================================
 FOSS Package Inspector: bash
-------------------------------------------
 STATUS  : INSTALLED
 Version : 5.2.21-2ubuntu4
 Summary : GNU Bourne Again SHell
 License : See /usr/share/doc/bash/copyright
-------------------------------------------
 Open Source Philosophy Note:
 'bash': free to use, study, and share.
===========================================
```

Script 3 — Disk and Permission Auditor

Usage: bash script3_disk_permission_auditor.sh

Loops through an array of seven important Linux system directories and for each one that exists prints its permission string, owner, group, and disk usage in a formatted table. Then performs a dedicated check on /proc/sys/kernel and prints three live kernel tunable values as evidence.

Shell concepts demonstrated:

- Array — DIRS=("/etc" "/var/log" ...) defines an indexed array. "${DIRS[@]}" expands all elements safely.
- for loop — for DIR in "${DIRS[@]}" iterates over each element in sequence. The fundamental looping construct for processing a list.
- - d test — [-d "$DIR"] returns true only if the argument is a directory that exists. The else branch handles missing directories without crashing.
- awk — ls -ld "$DIR" | awk '{print $1}' extracts the permissions field. Fields $3 and $4 give owner and group.
- du -sh and cut — du -sh "$DIR" | cut -f1 gets human-readable size and strips the path that du appends after a tab.
- printf — used for the table header to align columns with fixed-width format specifiers (%-20s, %-12s).

Complete annotated script:

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: TANISHKA SHARMA Course: Open Source Software

# --- Array of system directories to audit ---
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/proc" "/sys")
KERNEL_CONFIG_DIR="/proc/sys/kernel"

echo "================================================================"
echo " Directory Audit Report -- $(date '+%d %B %Y %H:%M:%S')"
echo "================================================================"
```

```
# printf aligns columns with fixed-width format specifiers
printf "%-20s %-12s %-10s %-8s %s\n" \
       "Directory" "Permissions" "Owner" "Group" "Size"
echo "-----------------------------------------------------------------"

# --- for loop: iterate over each element of the array ---
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld "$DIR" | awk '{print $1}')
        OWNER=$(ls -ld "$DIR" | awk '{print $3}')
        GROUP=$(ls -ld "$DIR" | awk '{print $4}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        printf "%-20s %-12s %-10s %-8s %s\n" \
               "$DIR" "$PERMS" "$OWNER" "$GROUP" "${SIZE:-N/A}"
    else
        printf "%-20s %s\n" "$DIR" "[ does not exist ]"
    fi
done

# --- Dedicated kernel config directory check ---
if [ -d "$KERNEL_CONFIG_DIR" ]; then
    echo ""
    echo " Kernel Config Dir: $KERNEL_CONFIG_DIR"
    PERMS=$(ls -ld "$KERNEL_CONFIG_DIR" | awk '{print $1}')
    echo " Permissions: $PERMS (world read-only; root write)"
    for TUNABLE in hostname ostype pid_max; do
        echo "   $TUNABLE = $(cat $KERNEL_CONFIG_DIR/$TUNABLE 2>/dev/null)"
    done
fi
echo "=================================================================="
```

Live terminal output (Ubuntu 24.04.4 LTS, confirmed):

```
==================================================================
 Directory Audit Report -- 28 March 2026 10:35:49
==================================================================
Directory            Permissions  Owner      Group    Size
------------------------------------------------------------------
/etc                 drwxr-xr-x   root       root     5.8M
/var/log             drwxr-xr-x   root       root     981K
/home                drwxr-xr-x   root       root     2.7G
/usr/bin             drwxr-xr-x   root       root     436M
/tmp                 drwxrwxrwt   root       root     24M
/proc                dr-xr-xr-x   root       root     0
/sys                 drwxr-xr-x   root       root     0

Kernel Config Dir: /proc/sys/kernel
Permissions: dr-xr-xr-x (world read-only; root write)
   hostname = runsc
   ostype = Linux
   pid_max = 65536
==================================================================
```

Script 4 — Log File Analyser

Usage: bash script4_log_analyser.sh /var/log/dpkg.log installed

Takes a log file path as $1 and optional keyword as $2 (defaulting to 'error'). Reads the file line by line, counting every line containing the keyword. Prints total lines read, match count, percentage,

and the last five matching lines. Demonstrated live on /var/log/dpkg.log with keyword 'installed': 2,123 matches in 8,185 lines (25% match rate).

Shell concepts demonstrated:

- Command-line arguments $1, $2 — $1 receives the filename, $2 the keyword. ${2:-"error"} provides a default value using shell parameter expansion if $2 is unset.
- - z and -f tests — [-z "$LOGFILE"] checks for a missing argument; [! -f "$LOGFILE"] verifies the file exists. Both trigger early exit with a clear message.
- while IFS= read -r LINE — the standard idiom for reading a file line by line. IFS= prevents whitespace stripping; -r prevents backslash interpretation; < "$LOGFILE" feeds the file into the loop's stdin.
- Counter variable — COUNT=$((COUNT + 1)) uses arithmetic expansion. Shell arithmetic requires $((...)) syntax.
- Do-while style retry — while [! -s "$LOGFILE"] implements a retry loop for empty files with a MAX_RETRIES guard.
- grep | tail — grep -i "$KEYWORD" "$LOGFILE" | tail -5 finds all matching lines and keeps only the last five.

Complete annotated script:

```
#!/bin/bash
# Script 4: Log File Analyser
# Author: TANISHKA SHARMA Course: Open Source Software
# Usage: ./script4_log_analyser.sh <logfile> [keyword]

LOGFILE=$1
KEYWORD=${2:-"error"}   # default if $2 not provided
COUNT=0 ; TOTAL=0

# --- Validate arguments ---
if [ -z "$LOGFILE" ]; then
    echo "Usage: $0 <logfile> [keyword]"; exit 1
fi
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File '$LOGFILE' not found."; exit 1
fi

echo "================================================================"
echo " Log File Analyser | File: $LOGFILE | Keyword: '$KEYWORD'"
echo "================================================================"

# --- Do-while style: retry if file is empty ---
MAX_RETRIES=3; RETRIES=0
while [ ! -s "$LOGFILE" ]; do
    RETRIES=$((RETRIES + 1))
    [ "$RETRIES" -ge "$MAX_RETRIES" ] && echo "Giving up." && exit 1
    read -p " File is empty. Enter alternative path: " LOGFILE
done

# --- Main loop: read file line by line ---
while IFS= read -r LINE; do
    TOTAL=$((TOTAL + 1))
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo " Total lines read   : $TOTAL"
echo " Lines with keyword : $COUNT"
[ "$TOTAL" -gt 0 ] && echo " Match rate         : $((COUNT * 100 / TOTAL))%"
echo ""
```

```
echo " Last 5 matching lines:"
echo "----------------------------------------------------------------"
grep -i "$KEYWORD" "$LOGFILE" | tail -5 | while IFS= read -r M; do
    echo "  >> $M"
done
echo "================================================================"
```

Live terminal output (Ubuntu 24.04.4 LTS, confirmed):

```
================================================================
 Log File Analyser | File: /var/log/dpkg.log | Keyword: 'installed'
================================================================
 Total lines read   : 8185
 Lines with keyword : 2123
 Match rate         : 25%

Last 5 matching lines:
----------------------------------------------------------------
  >> 2026-02-18 19:59:13 status installed gstreamer1.0-plugins-bad
  >> 2026-02-18 19:59:14 status installed dictionaries-common:all
  >> 2026-02-18 19:59:14 status installed libc-bin:amd64 2.39
  >> 2026-02-18 20:00:19 status installed ca-certificates-java
  >> 2026-02-18 20:00:22 status installed ca-certificates-java
================================================================
```

Script 5 — Open Source Manifesto Generator

Usage: bash script5_manifesto_generator.sh (interactive — answer three prompts)

Prompts the user interactively with three questions using read -p. Validates that no answer was left blank. Composes a personalised open source philosophy statement using the three answers, writes it to manifesto_<username>.txt using > and >>, and displays it with cat.

Shell concepts demonstrated:

- read -p — displays a prompt string and waits for user input. The typed value is stored in the named variable (TOOL, FREEDOM, BUILD). This is the fundamental interactive input mechanism in bash.
- String concatenation — "manifesto_${AUTHOR}.txt" joins a literal string with a variable. The ${} form delimits the variable name clearly from surrounding text.
- File writing with > and >> — > creates or overwrites the output file. Every >> appends without erasing prior content. The manifesto is assembled line by line.
- date command — $(date '+%d %B %Y') captures the current date. %d = day, %B = full month name, %Y = four-digit year.
- Alias concept (demonstrated in comments) — alias today='date +%d\ %B\ %Y' would work interactively. Scripts use command substitution since aliases do not export to subshells.
- -z validation — [-z "$TOOL"] checks that the user did not leave the field blank before composing the manifesto.

Complete annotated script:

```
#!/bin/bash
```

```
# Script 5: Open Source Manifesto Generator
# Author: TANISHKA SHARMA Course: Open Source Software
# Alias concept (interactive shells only; scripts use $() instead):
# alias today='date +%d\ %B\ %Y'

DATE=$(date '+%d %B %Y')
AUTHOR=$(whoami)
OUTPUT="manifesto_${AUTHOR}.txt"   # string concatenation

echo "================================================================"
echo " Open Source Manifesto Generator"
echo " Your manifesto will be saved to: $OUTPUT"
echo "================================================================"
echo ""

# --- read -p: interactive input ---
read -p " 1. Name one open-source tool you use every day: " TOOL
echo ""
read -p " 2. In one word, what does 'freedom' mean to you? " FREEDOM
echo ""
read -p " 3. Name one thing you would build and share freely: " BUILD
echo ""

# --- -z validation: ensure no answer was left blank ---
if [ -z "$TOOL" ] || [ -z "$FREEDOM" ] || [ -z "$BUILD" ]; then
    echo " Error: All three answers are required."; exit 1
fi

# --- Write to file: > creates, >> appends ---
echo "================================================================" > "$OUTPUT"
echo " OPEN SOURCE MANIFESTO" >> "$OUTPUT"
echo " Written by : $AUTHOR" >> "$OUTPUT"
echo " Date       : $DATE" >> "$OUTPUT"
echo "================================================================" >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo " I believe software should serve people, not trap them." >> "$OUTPUT"
echo " Every day I rely on $TOOL -- built for the community." >> "$OUTPUT"
echo " To me, freedom means $FREEDOM: the right to read the" >> "$OUTPUT"
echo " code on my machine, change what doesn't work, and" >> "$OUTPUT"
echo " pass improvements on to others." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo " Torvalds didn't keep Linux proprietary. Stallman didn't" >> "$OUTPUT"
echo " lock away the GNU tools. They shared because knowledge" >> "$OUTPUT"
echo " grows when given away. One day I will build $BUILD" >> "$OUTPUT"
echo " and release it freely -- because the ecosystem I benefit" >> "$OUTPUT"
echo " from deserves something back." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo " Signed: $AUTHOR | $DATE" >> "$OUTPUT"
echo "================================================================" >> "$OUTPUT"

echo " Manifesto saved to: $OUTPUT"
cat "$OUTPUT"
```

Live terminal output (Ubuntu 24.04.4 LTS, confirmed):

```
================================================================
 OPEN SOURCE MANIFESTO
 Written by : root
 Date       : 28 March 2026
================================================================

I believe software should serve people, not trap them.
 Every day I rely on Linux Terminal -- built for the community.
 To me, freedom means Choice: the right to read the
 code on my machine, change what doesn't work, and
 pass improvements on to others.
```

```
Torvalds didn't keep Linux proprietary. Stallman didn't
 lock away the GNU tools. They shared because knowledge
 grows when given away. One day I will build A free OS
 and release it freely -- because the ecosystem I benefit
 from deserves something back.

Signed: root | 28 March 2026
================================================================
```